

Industrial Training – Project Report
(MCA-C651)
on
SMART INVENTORY MANAGEMENT SYSTEM

*Submitted in partial fulfillment of the requirements
for the award of the degree of*

Master of Computer Applications (MCA)
Uttaranchal School of Computing Sciences



**UTTARANCHAL
UNIVERSITY
DEHRADUN**

**NAAC
GRADE A+**

BATCH 2024-2026

Submitted to:

Dr. Manoj Aswal

Assistant Professor

Uttaranchal School of Computing Sciences

Submitted by

Ayushi Khandelwal

UU2420000021

MCA -IV (A)

ACKNOWLEDGEMENT

The most awaited moment of any endeavor is its successful completion, but nothing can be done successfully if done alone. Success is the outcome of continuous technical mentoring, guidance, and support from various individuals, and I express my sincere gratitude to all those who contributed to the successful completion of this project.

First and foremost, I would like to thank **Prof. (Dr.) Sonal Sharma, Director, USCS**, for providing a healthy and encouraging environment for study.

I am extremely thankful to **Prof.(Dr.) Sameer Dev Sharma, Head of Department, USCS**, and my project mentor **Dr. Manoj Aswal**, for providing me with this opportunity and for their valuable technical mentorship, continuous guidance, and expert insights throughout the project development process. Their support at every stage significantly enhanced my understanding and helped me successfully complete this work.

I also express my sincere gratitude to my colleagues for their valuable discussions and support, which helped me in giving a final shape to this project

Dated:

Ayushi Khandelwal

UU2420000021

MCA – IV(A)

STUDENT DECLARATION

I hereby declare that the project titled “**Smart Inventory Management System**” is an original work carried out by me under the guidance of **Dr. Manoj Aswal** in partial fulfillment of the requirements for the award of the degree of **MCA** from **Uttaranchal University**.

I further declare that this project has not been submitted, either in part or in full, to any other institution or university for the award of any degree or diploma.

All sources of information and references used in this project have been duly acknowledged.

Dated:

Ayushi Khandelwal

UU2420000021

MCA – IV(A)

Uttaranchal School of Computing Sciences

CERTIFICATE

This is to certify that the project work entitled “**Smart Inventory Management System**” is a bonafide work carried out by **Ayushi Khandelwal (UU2420000021)** submitted in partial fulfilment for the award of the degree in **MASTER OF COMPUTER APPLICATIONS** of **Uttaranchal University, Dehradun** during the academic year 2025-26.

It is verified that the project files and project report (PDF) are uploaded on GitHub [<https://github.com/ayushiKhandelwal24/Smart-Inventory-Management-System>]

Dr. Manoj Aswal

Assistant Professor

USCS

INDUSTRIAL TRAINING CERTIFICATE



Certificate of Internship

Is proudly awarded to

Ayushi Khandelwal



This is to certify that Ms. Ayushi Khandelwal has successfully completed an internship with Plasmid in the domain of web development from January 15th 2026 to April 15th 2026. Her performance in the program was commendable and her efforts were found to be sincere and diligent.

K Praveen Kumar,
Senior Manager,
Operations & HR

ID : 1124CA15647

PREFACE

This project report titled “[**SMART INVENTORY MANAGEMENT SYSTEM**]” has been prepared as a partial fulfilment for the award of the degree of **Master of Computer Applications**. The objective of this project is to apply theoretical concepts to real-world problems and to gain practical exposure in design, development, and implementation

[The Smart Inventory Management System is a web-based application designed to solve issues in manual inventory handling such as stock mismanagement and lack of real-time tracking. The objective is to automate product management, monitor stock levels, and generate reports efficiently. The system is developed using Flask (Python), MySQL, and HTML/CSS/JavaScript. It includes features like user login, shop selection, product CRUD operations, low stock alerts, and CSV report export. The key outcome is improved accuracy, reduced manual effort, and better decision-making through real-time data management.]

The project files are uploaded on GitHub [<https://github.com/ayushiKhandelwal24/Smart-Inventory-Management-System>]

Dr. Manoj Aswal
Assistant Professor

Ayushi Khandelwal
UU2420000021

TABLE OF CONTENTS

Contents	Page no.
* Acknowledgement	i
* Declaration	ii
* Certificate	iii
* Industry Training Certificate	iv
* Preface	v
1. Introduction / Objective	1-4
1.1 Introduction	1-2
1.2 Objective	3-4
2. System Analysis	5-23
2.1 Identification of Needs	5
2.2 Preliminary Investigation	5
2.3 Feasibility Study	5-6
2.4 Project Planning	7-8
2.5 Project Scheduling (Pert Chart And Gantt Chart)	9-10
2.6 Software Requirement Specification (SRS)	11-13
2.7 Software Engineering Paradigm Applied	13-14
2.8 Data Models And System Diagrams	15-23
2.8.1 Data Flow Diagram (DFD) Level 0,1,2	15-17
2.8.3 Use - Case Diagram	18
2.8.4 Class Diagram	18-19
2.8.5 Sequence Diagram	20-23
3. System Design	24-31
3.1 Modularisation Details	24-25

3.2 Data Integrity And Constraints	25-27
3.3 User Interface Design	27-31
4. Testing	32-35
4.1 Objective of Testing	32
4.2 Types of Testing Performed	32-33
4.3 Test Cases	33-35
5. System Security Measure	36-37
6. Cost Estimation and cost Estimate Model	38-40
7. Reports	41-45
8. Future Scope And Enhancement	46-48
9. Appendices	49-69
9.1 Code	49-68
9.2 Bibliography	69

1.INTRODUCTION AND OBJECTIVES

1.1 Introduction

Smart Inventory Management System is a web application created to help manage the level of stock in various retail shops. It was built using Python Flask framework and the MySQL relational database. The application helps to manage stock levels in various shops and offers automated intelligence about stock levels in different shops. In an era of data-driven decisions making, the application comes at a time when manual ledgers used to keep track of stock levels in retail shops are outdated.

Inventory management is critical to running any business whether a retail or a manufacturing company. Usually, manual ledger systems or simple spreadsheets are enough in keeping track of inventory for small to medium businesses. However, as these businesses grow in terms of their size and begin to carry out their functions not only from one business, but from multiple stores, then the above mentioned approaches do not work any more. Stock excess and stock shortage are the two major challenges that face such organizations.

The primary challenges addressed by this project include:

Data Inconsistency: Manual tracking often leads to discrepancies between physical stock and recorded stock.

Lack of Real-time Monitoring: Business owners often realize a product is out of stock only when a customer requests it.

Among the main problems addressed in this project includes:

Discrepancies between Actual Stock Levels and the Data in Inventory Ledgers.

Objectives of Smart Inventory Management Systems are:

1. To automate CRUD Operations – to give smooth operations to Add, View, Edit and Delete inventory items.

2. To implement Multi-Shop feature – by using Foreign Keys in MySQL to manage different inventories for multiple independent shops.
3. To provide Smart Notification – developing a logic that triggers notification to the user when there are less than a certain amount of stock (e.g., $\$ < 5\$$ stock).
4. To increase Data portability – giving permission to administrators to export real-time data to CSV file format.
5. To ensure data security – implementing session management and login for protection of business secrets.

Scope of our project includes building a fully-functional back-end with Python programming language, front-end with HTML, CSS and Bootstrap, and database layer with MySQL. Although this system runs on a localhost currently, its cloud-ready approach will make sure the system scales well with increasing number of users. This system can be used in retail industry where updating and visibility of low stocks are the priority.

Our system will be built on the Waterfall approach in software development which includes Requirement Gathering, Analysis, System design (using DFD and UML diagrams), Coding and Testing.

1.2 Objectives

The following objectives are the main drivers behind the implementation of the Smart Inventory Management System:

1.2.1 Primary Objective

The main objectives include those which define basic functionalities of the system:

- To Centralize Inventory Monitoring: To implement an online tool that would allow users to keep track of stocks across different shop locations in a single web interface rather than using different software or books.
- To Integrate User Authentication: To implement secure login credentials using Flask session management which would allow accessing inventory information only to certain authorized users.
- To Implement Real-time CRUD Functionality: To enable creation, reading, updating and deleting of any record in the inventory immediately reflecting changes in the warehouse/shop floor.
- To Implement Multiple Shop Relations Using Foreign Keys: To implement a "one-to-many" relation between different shops and their goods based on MySQL foreign keys to make data well-structured.

1.2.2 Specific Objectives

- The additional goals include those which define "smart" functions of the system:
- To Implement Automated Notifications: To develop algorithms of comparing stocks' amount to certain threshold and to notify users about shortage using red alerts.
- To Make the Software Portable and Professional: To implement functionality of creating reports which convert live MySQL data into CSV files. To Optimize User Experience (UX): To utilize Bootstrap to create a responsive, clean, and intuitive frontend that works seamlessly across desktop and tablet browsers.
- To Prevent Data Loss and Injection: To implement secure coding practices, such as Parameterized Queries, to protect the database from common vulnerabilities like SQL Injection.

This sub-section explains *why* a shop would want your software:

1. Reduction in Human Error: By automating calculations and data entry, the system minimizes the risk of mathematical mistakes found in manual logs.
2. Optimized Capital Investment: By identifying slow-moving items and low-stock items, the owner can invest money in products that actually sell.

3. Scalability: The system is designed to grow; as the business opens new branches, the admin can simply add a new shop entry in the database without changing the code.

2. SYSTEM ANALYSIS

Systems Analysis refers to the activity of collecting, analyzing facts, problem-solving, and recommending system improvements based on these findings.

2.1 Identification of Need

The need for the system was identified through the various limitations in the manual/spreadsheet inventory management methods.

- **Error Rate:** High dependence on the human memory and manual registration resulted in high error rates.
- **Scalability Limitation:** With expansion to more than one shop branch, it would be difficult for the system to track the items available in all the branches unless there is a central database.
- **Selling Items Out:** The system needed to provide alerts to the owner about an item that is almost getting sold out.
- **Protection Against Theft/Manipulation:** Data in manual form is easily susceptible to damage and theft.

2.2 Preliminary Investigation

The preliminary survey was conducted to set the limits of the project.

- **Facts Collection:** Interviews with the shop owners indicated that a majority of them spend 2-3 hours per day to reconcile their stock.
- **Current Analysis of Systems:** The current inventory management systems are either too costly or not flexible enough.

2.3 Feasibility Study

This study evaluates whether the project should proceed based on three core dimensions:

2.3.1 Technical Feasibility

The project uses the Python-Flask stack and MySQL. They are all open source, highly documented, and run efficiently on regular hardware. As the development team has the ability to use these technologies, the project is technically feasible.

2.3.2 Economic Feasibility

The cost of development is negligible, as it is based on open source technology (VS Code, Python, MySQL). The "Return on Investment" (ROI) is high, as the system saves costs by reducing labor and avoiding losses due to out-of-stock items.

2.3.3 Operational Feasibility

The application has a user-friendly interface developed with Bootstrap. It needs minimum training for the administrator to operate, and hence is operationally feasible even for non-tech-savvy shop owners.

It specifies the requirements of the environment where the development and execution of the program will take place.

3.4.1 Hardware Requirements

- CPU: Intel Core i3 or better (to facilitate smooth local server operations).
- Memory: 4 GB RAM (8 GB or more suggested for efficient simultaneous use of IDE and MySQL).
- Storage Space: 256 GB HDD/SSD (minimum 1 GB space available for storing data and log files).
- Input Device: Regular Keyboard and Mouse.

3.4.2 Software Requirements

Operating System: Windows 10/11, macOS, and Linux.

- Programming Language: Python 3.10 or newer.
- Web Framework: Flask 2.x or newer. Database Management System: MySQL Server 8.0+.
- Frontend: HTML5, CSS3, JavaScript, and Bootstrap 5.
- IDE: Visual Studio Code (VS Code) or PyCharm.
- Web Browser: Google Chrome, Mozilla Firefox, or Microsoft Edge.

2.4 Project Planning

The project planning of Smart Inventory Management System was made in a systematic way so that the project could be completed on time in an efficient manner. The first step included defining the scope of the project and listing down the objectives such as authentication of users, product management, inventory management, alert system, and reporting feature. Then a proper timeline of the project was established by splitting it into different phases like requirement analysis, system design, implementation, testing, and deployment of the project. The resources required for the project were also listed like development tools, database system, and hardware components. Scheduling of tasks and proper allocation of work were done so that efficient flow of work could take place. Risks and mitigation techniques were also considered while planning the project.

Task ID	Activity Description	Predecessor (Dependencies)	Estimated Duration
A	Requirement Analysis & Problem Definition	None	1 Week
B	Database Schema Design (MySQL)	A	1 Week
C	Backend Logic Development (Flask)	B	1.5 Weeks
D	Frontend UI Design (HTML/CSS)	B	1 Week
E	Feature Integration (Alerts & Export)	C, D	1 Week
F	System Testing & Debugging	E	0.5 Weeks
G	Final Documentation & Submission	F	0.5 Weeks

2.5 Project Scheduling (Pert chart and Gantt Chart)

Pert chart

Pert Chart – project flow and dependencies

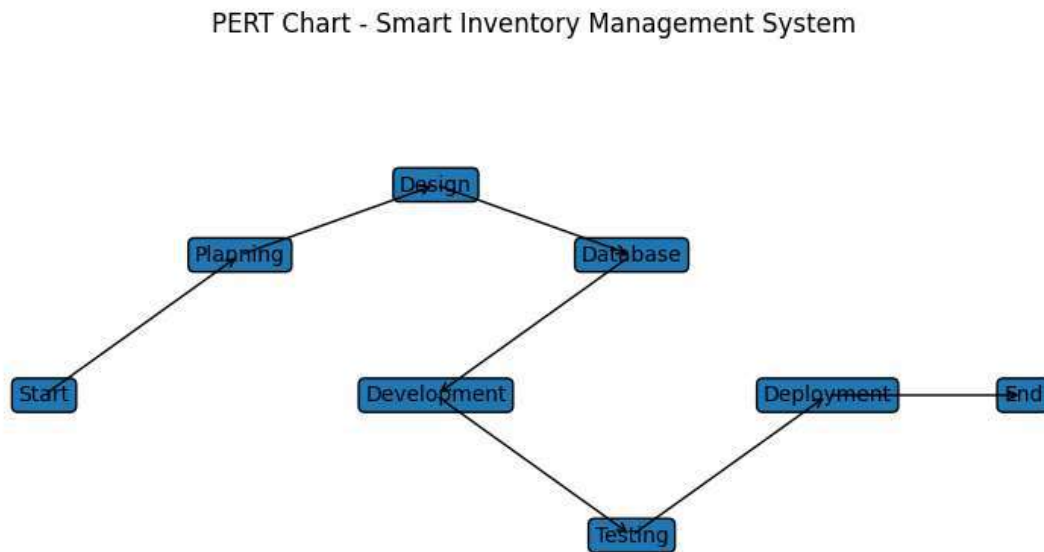


Fig-1 Pert Chart

Gantt Chart - project schedule for smart inventory management system

Task ID	Task Description	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6
T1	Requirement Analysis	■ ■					
T2	Database Design (MySQL)		■ ■				
T3	Backend Development (Flask)			■ ■			
T4	Frontend (HTML/CSS)				■ ■		
T5	Feature Integration & Testing					■ ■	
T6	Documentation & Submission						■ ■

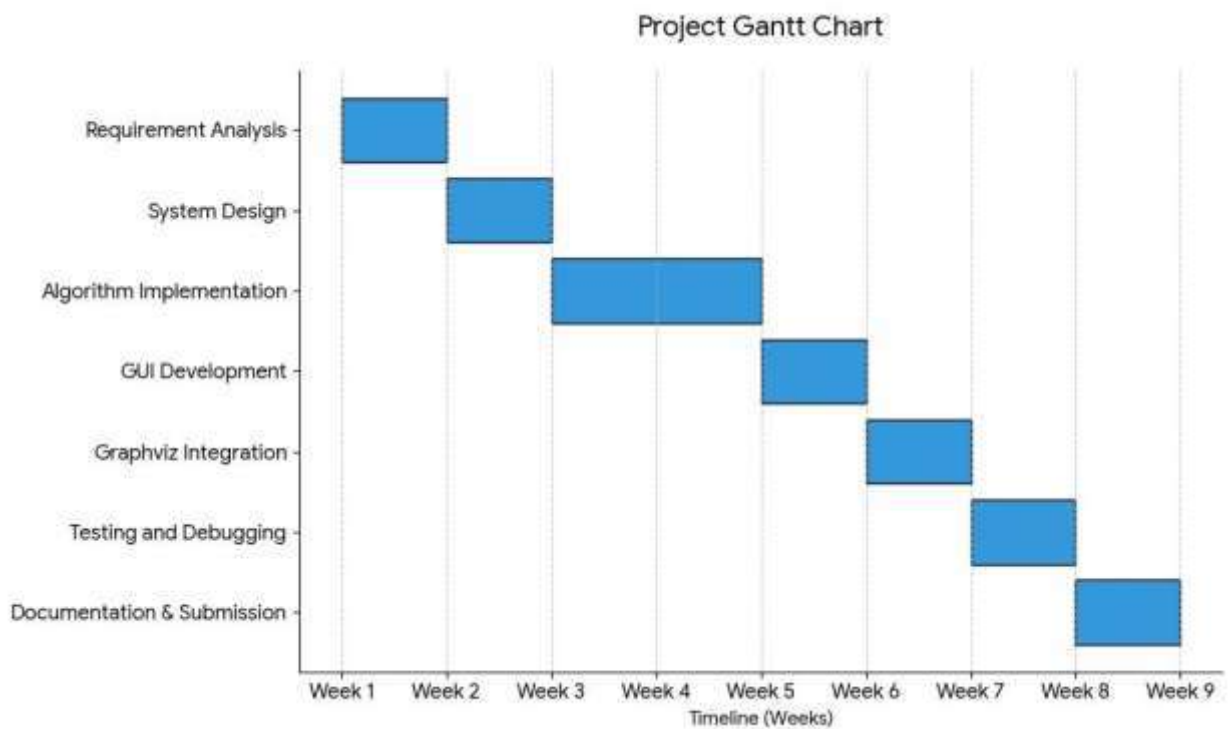


Fig 2 Gantt chart

2.6 Software Requirement Specification (SRS)

Smart Inventory Management System is a web application system which can manage inventory processes efficiently. This application system works on the client-server model where there is interaction between the front-end and back-end as well as back-end interacts with the database.

Users have the ability to perform various activities like managing products, inventory checks, and report making. In addition, real-time processing is made available and consistent data is ensured.

Multiple stores have been supported by this application system, each having their data related to inventory. Multiple inventories can be managed through this system.

2.6.1 Functions of the Product

The system has the following functions:

- Provides secured login services to authenticated users
- Provides features for selecting and managing various shops
- Facilitates addition, modification, and deletion of product entries
- Offers real-time inventory tracking service
- Gives alert services for stock shortages
- Facilitates reports generation and exporting features
- Supports proper data storage and retrieval services from the database

2.6.2 User Attributes

The system is meant for admin users who control inventory information.

Admin User

In charge of controlling shops and products

CRUD-capable

Computer-savvy in terms of basics

Familiar with web-based application

It is expected that the user has little technical prowess and can maneuver the system effortlessly.

2.6.3 Constraints

The system runs with the following constraints:

Stable internet connection is mandatory

Dependence on MySQL database

Web-based application only

Backend environment must be properly configured

Server capacity is a performance constraint

Basic security measures (can be improved)

2.6.4Functional

The Smart Inventory Management System provides functionalities such as user authentication, shop selection, and product management including adding, updating, deleting, and viewing products. It allows real-time inventory tracking, generates low stock alerts when quantity falls below a threshold, and enables users to export inventory data in CSV format for reporting purposes.

2.6.5Non-Functional

The system ensures fast response time and efficient performance while handling user requests. It provides secure access through authentication and protects data from unauthorized use. The interface is user-friendly and easy to navigate, ensuring reliability and consistency in operations. The system is scalable for future enhancements and compatible with standard web browsers, with proper error handling mechanisms in place.

2.6.6 External Interface Requirements:

Smart Inventory Management System uses web-based user interface, with login page, dashboard, product management, shop selection, and reporting page as the main pages. The system requires basic hardware such as computer or laptop connected to the internet. For database interaction, the system connects to the MySQL database using Flask (Python). The web browser provides a smooth interaction between the system and the users.

2.6.7 Data Requirement:

The system needs to take data inputs such as username and password for login, name, number of items, cost per item, and threshold level for product. The system then manipulates the data and produces outputs such as listing of stock in hand, stock warning messages, and report

generation in CSV file formats. Data is stored in a relational database structure in tabular forms of users, shops, and products.

2.7 Software Engineering Paradigm

The software engineering paradigm, on the other hand, involves how the process of development of the software system should be organized. Several phases exist in such a development process including those of requirements analysis, design, implementation, testing, and maintenance of the software system. Choosing the appropriate paradigm is essential for making sure that the process of software development is done in an efficient manner.

With regard to the development of the Smart Inventory Management System, a systematic and structured paradigm has been chosen.

Waterfall Model Phases

2.7.1. Requirement Analysis

In this stage, all the requirements of the system were collected. All the functional aspects including the login aspect, product management, inventory management, alarm system, and report generation have been well described.

Requirements from the user have been collected to ensure that the system will fulfill its intended requirements.

2.7.2. System Design

System design involves designing the architecture of the system. All the components of the system have been developed as different modules including the authentication module, product management module, inventory management module, and report module.

Designing of the database has also been done, including tables like users, shops, and products. The system structure has been drawn through UML diagrams and data flow diagrams.

2.7.3. Implementation (Coding Phase)

Coding has been done in this phase where the back-end has been coded using Flask (Python). On the other hand, the front-end has been coded using HTML, CSS, and JavaScript.

The coding of each module has been done individually followed by their integration. Database connectivity has also been done through MySQL.

2.7.4. Test Phase

After implementing the system, tests were conducted on the system to see if all its functions operate correctly. Several tests were conducted such as unit tests, integration tests, and system tests.

Several problems found when conducting the test were sorted out for efficiency purposes.

2.7.5. Deployment Phase

After conducting successful tests, deployment of the system took place. The deployment of the system took place through the internet to the user.

2.8 Data Models and System Diagrams

This is what this section does, explaining the diagram that we created, giving us a clear idea about the functionality of the Huffman Coding System. It makes things much easier for us, as we get an idea about how Component A interacts with Component B.

2.8.1 Data Flow Diagram (DFD)

DFD Level 0

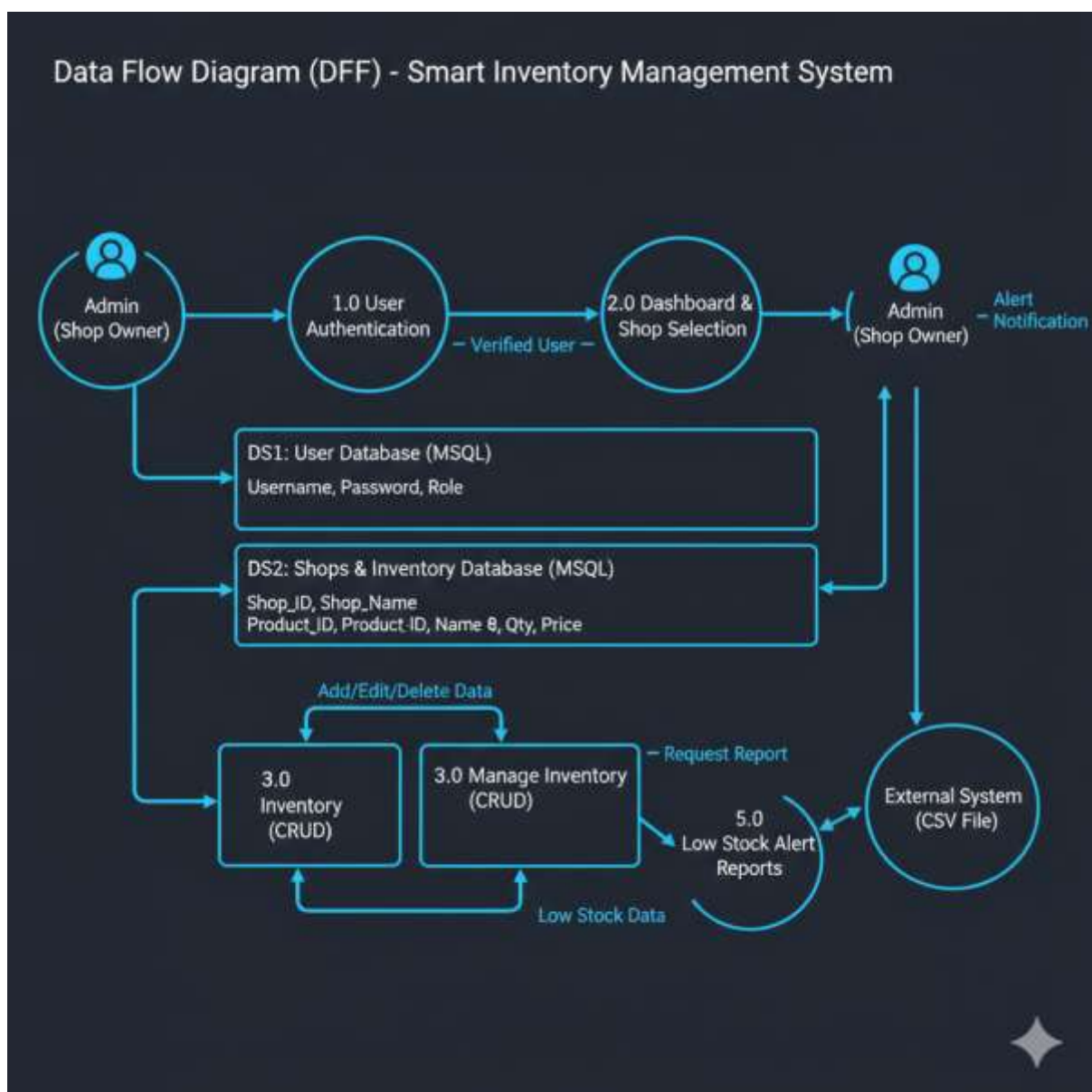


Fig3. DFD 0

DFD Level 1

Level 1 DFD shows how the system has been divided into processes with specific data flow between three primary components: Admin, Flask Logic and MySQL Database.

- Process 1.0 (Login): Verifies the credentials of admin using the Users table.
- Process 2.0 (Shop Selection): Gets data about the particular shop (for instance 'Electronics' or 'General Store') from the Shops table.
- Process 3.0 (Inventory Management): Works with CRUD functions that apply to products, and also updates the Products table with necessary information.
- Process 4.0 (Alert Logic): Searches through database tables for alerts regarding 'low stock products'.
- Process 5.0 (Data Export): Extracts data about certain shops from the database and exports it to CSV format.

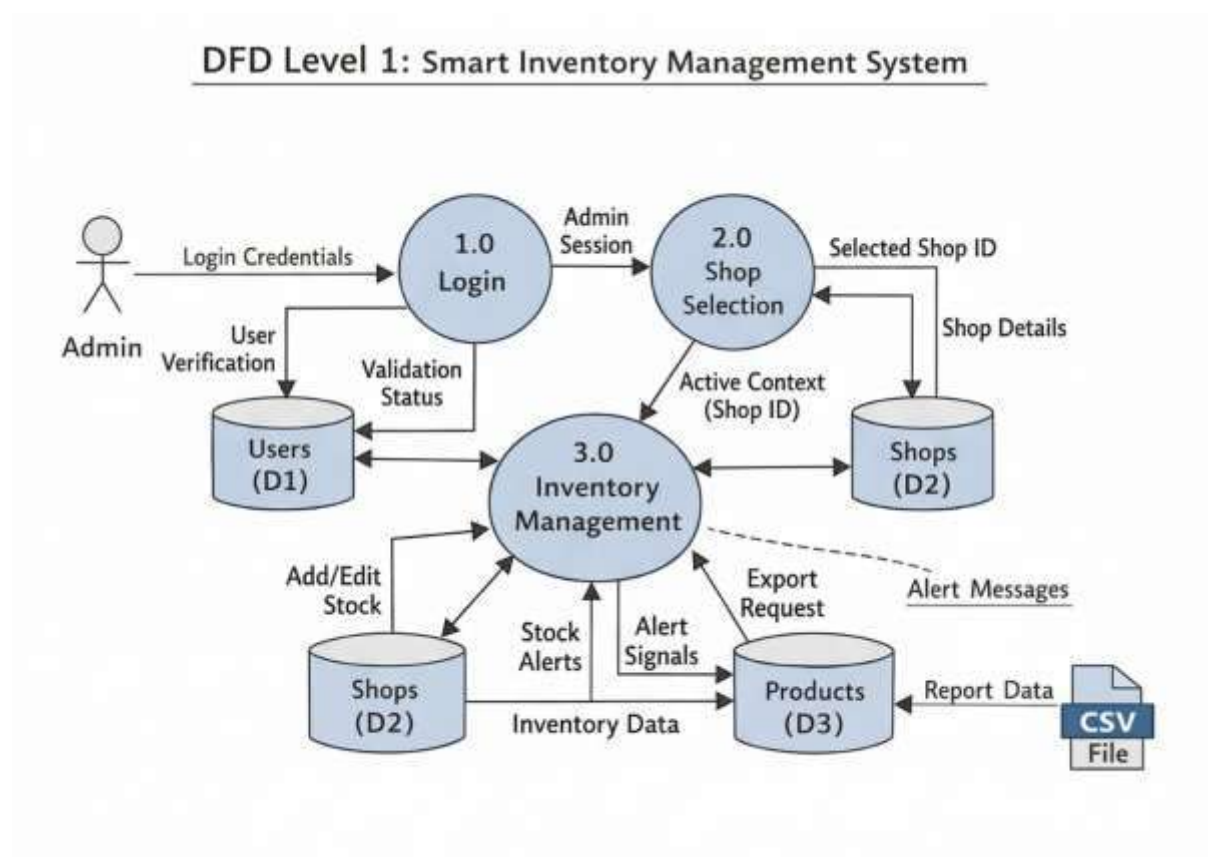


Fig 4 DFD Level 1

DFD Level 2

Description of the Decomposition of the General Process "Manage Inventory" of Level 2

Sub-Processes Decomposition

1. Subprocess 3.1 - Add Product Takes the data from the form "Add Product".

Verifies if the Name field and the Price field are present.

Saves the data using the Insert function in the Products table.

Subprocess 3.2 - Modify Existing Item

Takes the current entry by its Product_ID.

Modifies Quantity / Price fields.

Executes the Update function in the Database (MySQL).

Subprocess 3.3 - Remove Product Entry

Gets the delete command for the specified ID.

Removes the entry from the database, which deletes the product from the list.

Subprocess 3.4 – Get Entries of Specific Shop

Takes the entries from the database with Shop_ID as the parameter.

Checks if all the products are related to the specific store (General Store).

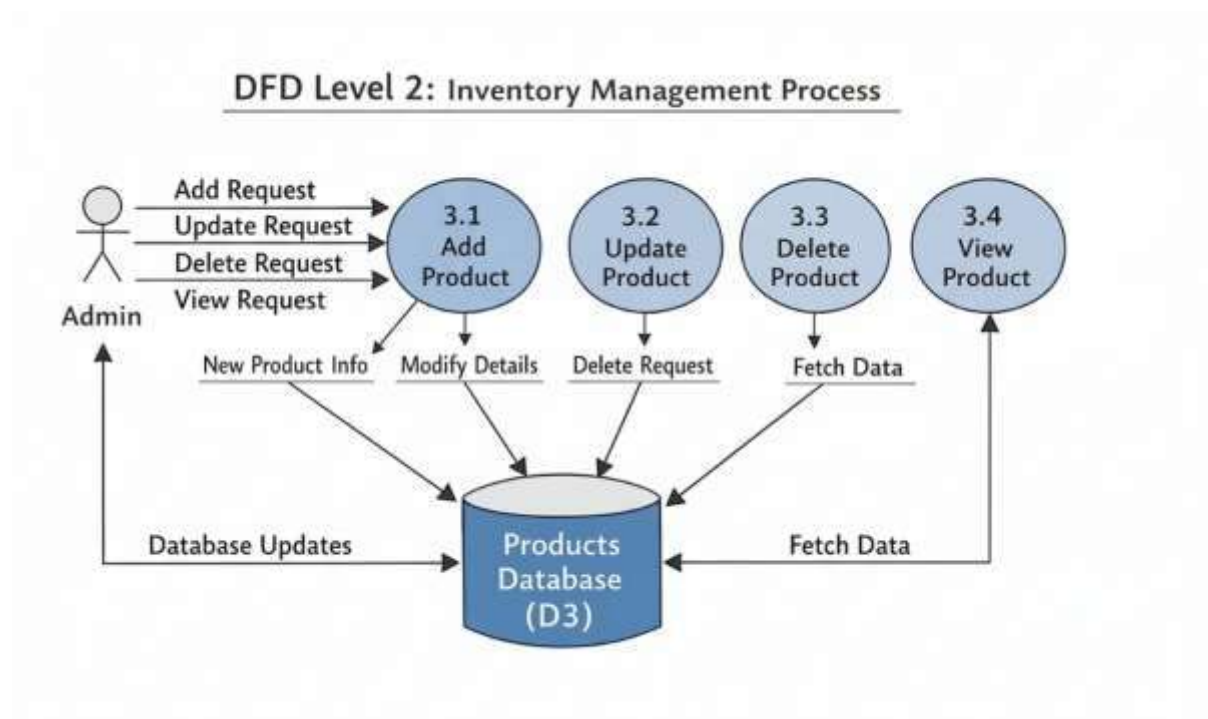


Fig5. DFD Level 2

2.8.3 Use-Case Diagram



Fig6. Use case diagram

2.8.4 Class Diagram

Your class diagram represent the inner workings of your python code (flask) and its relationship to the MySQL Database tables. The class diagram will define the attributes and methods of your application.

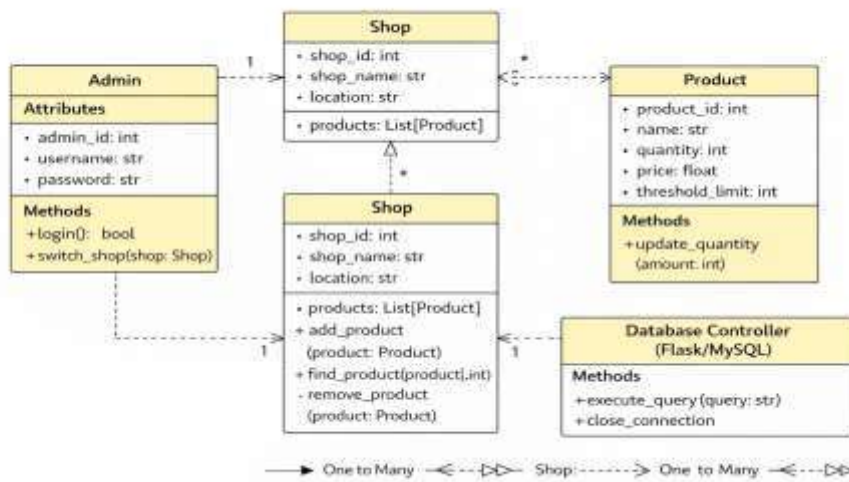


Fig7. Class Diagram

2.8.5 Sequence Diagram

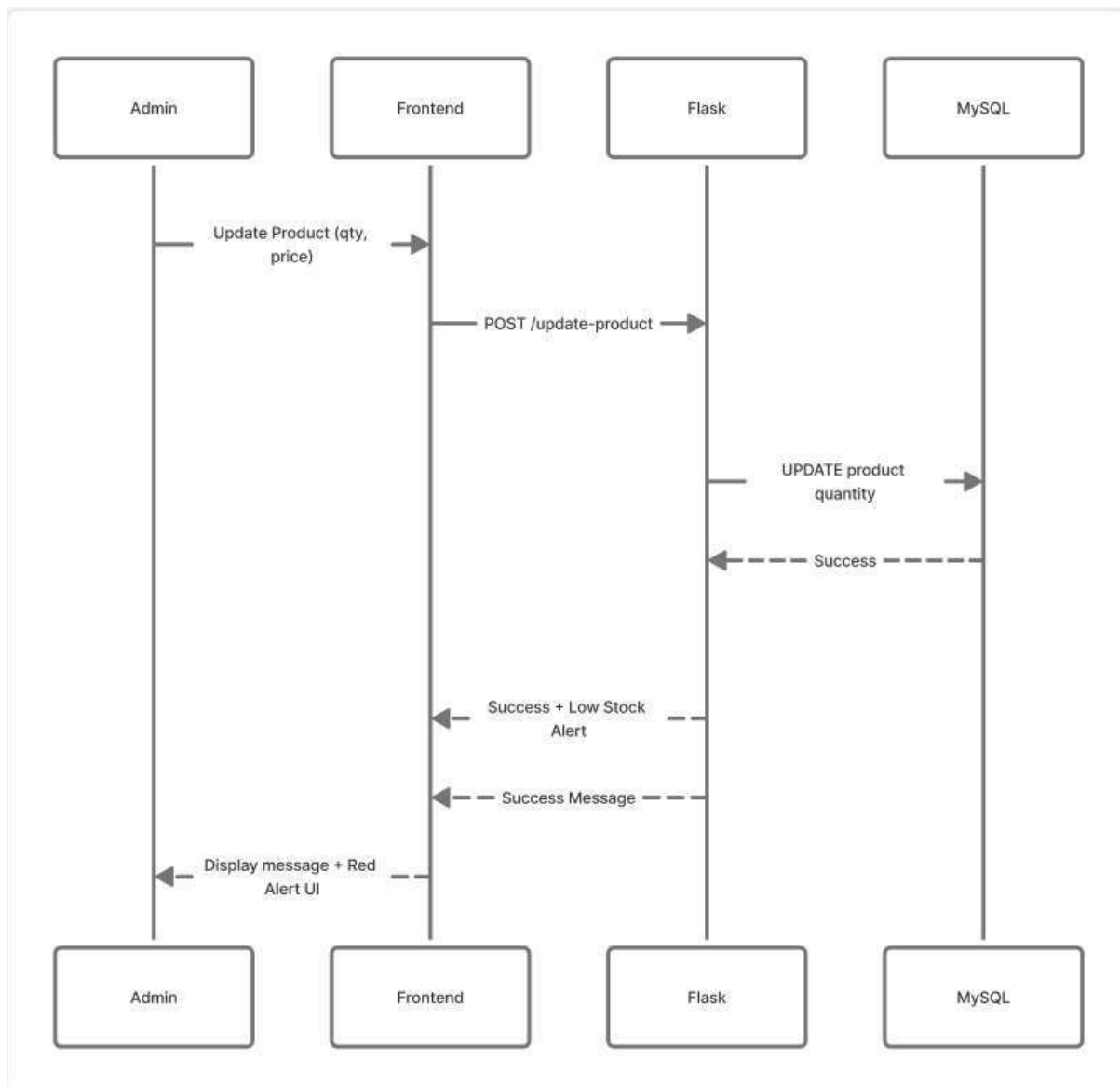


Fig8. Sequence Diagram

Application Icon: Symbolizing Smart Monitoring and Inventory Management System

- There is an icon of a box or storage symbolizing inventory/products
- There is also a graph or upwards arrow symbolizing growth and tracking
- A notification bell or alert symbol () symbolizing alert about low stock
- The color used includes:

Blue – representing reliability and technology

Green – growth and management

Red – alerts and notifications

The icon implies that this smart application is meant to monitor, manage and track inventories.

Key Features of Smart Inventory Management System

This application has key features that make it 'smart':

Automation

- Minimizes manual labor
- Updates inventories automatically

Accuracy

- Minimizes errors caused by human labor
- Guarantees data storage

Real-time Monitoring

- It updates users of inventory levels in real-time
- Quick decisions based on information

Alert Mechanism

- The system alerts when there is a low stock of products
- It prevents stock-outs

Scalability

- It works efficiently with multiple shops
- Efficiency even when working with bulk of data

Efficiency

- Saves both energy and time

- Streamlines processes

Process of Smart Inventory Management Application

It is simple but efficient. The process follows this systematic process:

Step One: User Log-In

Login using credentials by the administrator.

Step Two: Shop Selection

The administrator selects the shop where the inventory would be managed after logging into the application.

Step Three: Product Management

The administrator can perform the following tasks from here:

- Adding products
- Changing quantity of products
- Removing products
- Viewing all products

Step Four: Back End Processing

- Request and database interaction
- The back end, built using Flask, processes the request and connects to the database

Database Update Step Five

- Data updating and management would be done by the MySQL database

Stock Validation Step Six

- Check if stock of the product is less than the pre-defined limit

Alerting Step Seven

- If stock is not sufficient then:
- Send alert message
- Color the product (red)

Result Displaying Step Eight

Real-time display of results

Uses of Smart Inventory Management Systems

Some applications of smart inventory management systems are as follows:

Retail Stores

- For managing inventory every day
- To keep an eye on quickly selling items
- Preventing shortage of stock

Supermarket

- Keep track of different kinds of items

Warehouses

- To manage inventory being stored
- Inventory management for bulk purposes

Pharmacies

- Management of medicines' stock
- Basic medication availability

E-commerce Businesses

- Stock tracking
- Management of inventory at different locations

3.SYSTEM DESIGN

The importance of system design cannot be underestimated in terms of the development of the Smart Inventory Management System because it determines the general architecture of the system and interaction of all its components and database management.

The Smart Inventory Management System is designed using a layered approach and is modular by nature, providing scalability and efficiency. It consists of three layers.

- Presentation Layer (Front-end)
- Application Layer (Back-end – Flask)
- Data Layer (MySQL Database).

3.1 Modularisation Details

Modularity refers to the process of splitting a system into different modules. Although these modules function individually, they work together in unison such that the whole system works efficiently.

1. Authentication Module

2. Shop Management Module

This module is tasked with the creation, update, and management of different shops. Each shop has its own inventory in this system.

3. Product Management Module

This module is the most important module since it is used in managing the data of the products. You can use this module to create new products, update products, delete products, and check product information.

4. Inventory Management Module

This module keeps track of the number of stocks per product.

5. Alert Module

This module helps keep track of the number of stocks per product. This module raises an alarm when the stocks are below the expected levels.

6. Reporting Module

This module facilitates report generation. The data can be exported to the CSV format in this module.

- 7. Database Module
- This module will take care of all database management functions.

- 8. User Interface Module
- This module assists in creating the front end of the system.
- Benefits of Modularity
 - • More organized structure
 - • Easier debugging
 - • Scalability
 - • Concurrent development
 - • More maintainable

3.2 Data Integrity and Constraints

Data integrity refers to the reliability, validity, and accuracy of the data in the database. Constraints refer to the restrictions placed on the fields in the database.

Types of Data Integrity

1. Entity Integrity

- Each in the database contains a primary key
- There are no duplications
- E.g., Product IDs should not duplicate

2. Referential Integrity

- Keeps relationships between tables
- Checks that foreign key and primary keys match
- E.g., Each product should have a valid shop ID

3. Domain Integrity

- Checks that data fall within their domains
- E.g., Quantity can't be negative

4. User Defined Integrity

- Rules defined by the user
- E.g., Minimum stock level

Constraints Implemented in the Database

1. Primary Key Constraint

- The field identifies uniquely the records in the table
- E.g., product_id

2. Foreign Key Constraint

- This establishes relationships between two tables
- E.g., shop_id in products table

3. NOT NULL Constraint

- The field cannot contain null values
- E.g., Product name cannot be null

4. UNIQUE Constraint

- This ensures there is no duplication
- E.g., Product code

5. CHECK Constraint

- This checks the data entries for validity
- E.g., quantity ≥ 0

Importance of Data Integrity

- Ensures reliable data
- Avoids errors
- Enhances the performance of the system
- Fosters trust in the outputs

3.3 User Interface Design

User Interface (UI) refers to the portion of an interface that allows users to interact with the software application. Good design of the user interface enhances the use-friendliness of the software.

3.3.1 User Interface Design Guidelines Fulfilled

1.Simple UI

- Minimalistic
- Navigation-friendly

2. Consistent UI

- Standardized page layout
- Same color palette and font type

3. Responsive UI

- Functions properly on all screen types
- Works well on any device

4.Feedback UI

- Shows response messages
- Communicates about task completion or failure

3.3.2 User Interface Components

1. Sign In Page

- Username and password
- Sign in button

2. Dashboard

- Overview of inventory
- Navigation to application parts

3. Product Pages

- Create/edit/delete products
- Product list

4. Shop Selection Page

Managing Shops

4. Alert Display

Recommend low-stock products

Red color when in warning mode

5. User Interface Technologies

HTML base language

CSS style approach

JavaScript interaction technology

6. Relevance of User Interface Design

Optimizing user experience

Shortening learning duration

Increasing efficiency

Enhancing system appeal

"Smart" Notification Algorithms and Backend Logics
The crucial feature of the proposed system is its capability of transforming from a simple storage into an actively managed application due to development of specific backend algorithms written in Python and estimating the stock condition.

1. The Low-Stock Alert Algorithm

In this case, all intelligence of our system resides in threshold comparison logic.

- **Threshold Levels Setup:** All products have specific predefined threshold levels indicating the lowest amount of stock acceptable for that product and stored in the database.
- **Conditional Statement Execution:** During extraction of the inventory data, the Flask backend executes conditional statements for each product separately: if quantity < threshold.

- **Boolean Flag Setting:** Consequently, such products are set to specific flags allowing the rendering of corresponding notifications with help of Jinja2 frontend.

2. The Algorithm for Calculating Real-Time Inventory Value

In order to provide the administrator with the actual state of finances, we carry out the following operations:

Asset Calculation:

The total number of products in the inventory will be calculated according to the following formula: $\sum (\text{Quantity} \times \text{Price})$ for each product connected with the current shop_id.

Dynamic Calculation:

This way, it will be easy to recalculate the sum immediately as soon as the price or stock level changes.

3. Shop Multiple Filter through SQL Relation

To handle the data of more than one shop, the system implements One-to-Many relation principle:

Filtering the choice of shop ID in Flask Session,

All SQL queries will be filtered by WHERE shop_id = %s

Data Filtering:

This way we can prevent the manipulation and displaying of any other shops' data during the operation with an administration console of a specific shop, for example, while using "Shop B" the database cannot be accessed by "Shop A".

4. Report Generation and CSV Format Output

The report generation module helps to make the process very quick and convenient:

The latest data from MySQL Database is obtained,

"Smart" Notification Algorithms and Backend Logics

The core attribute of the proposed system is its potential for transitioning from a mere database storage system to an active management application. This feature becomes possible by developing certain backend algorithms written in Python that assess the condition of stock.

1. The Low-Stock Alert Algorithm

All the intelligence of this system is embodied in its threshold comparison logic.

- **Threshold Levels Set Up:** Every item possesses certain predefined threshold values stored in the database, which reflect its minimal allowable stock levels.

- Logical Operation Performed: While retrieving data from the inventory, the Flask backend runs conditional statements in respect of each product: if quantity < threshold.
- Setting Boolean Flags: As a result, those items will be flagged internally so that the Jinja2 frontend can render special visual alerts.

2. The Real-Time Inventory Value Calculating Algorithm

For providing the administrator with the current financial situation analysis, the following operations are performed:

Asset Calculation:

The sum of all products in the inventory is not saved as a fixed value but rather it is calculated with the formula $\sum (\text{Quantity} \times \text{Price})$ for each item which is associated with the current shop_id.

Dynamic Summation:

This will allow instant updating of the sum as soon as a product's price is changed or stock gets sold; no manual calculations are required to update the business's total asset at any time.

3. Multiple Shop Filtering by SQL Relation

- For working with data of multiple shops, the system follows the principle of "One-to-Many" relation:
- The shop ID selection is being recorded in the Flask Session,

And all further SQL requests are filtered via `WHERE shop_id = %s`

Data Filtering:

- This allows avoiding showing and manipulating any other shops' inventory while using an administration console of a particular shop, e.g., "Shop A" is never accessed and updated when working with "Shop B".

4. Report Generation and Data Saving as CSV File

- Data reports generation module makes the whole process easy and fast:
- The current data is fetched from the MySQL Database,

Next, using the help of the `io.StringIO` and `csv.writer` classes, the data gets converted into CSV (Comma Separated Value).

5. Input Sanitization & Error Handling

- To avoid crashing of the system owing to erroneous entries made by users, the following validations have been used:
- Casting to Specific Types: The user input values entered through the form are casted to integers or floats for compatibility with math.

- Check for Positive Numbers: It makes sure that the quantity and price are not negative, thus adhering to Domain Integrity.
- Positive numbers check: To ensure the entered numbers are not negative, hence implementing Domain Integrity.

4.TESTING

Testing is one of the critical processes in developing software applications since the system should perform efficiently without making any errors. Many forms of tests have been applied in implementing the Smart Inventory Management System.

4.1 Objectives of Testing

The primary objective of testing in this project is to make sure that all activities are performed correctly and efficiently. The tests aim at checking whether the encoding operation has been implemented correctly and the internal decoded outputs produced are accurate.

Tests also assist in ensuring that the system runs properly when different inputs are fed into the system, including those with empty and file inputs. Another goal of testing is to validate the performance of the application and check whether the graphical user interface works well.

4.2 Types of Testing Performed

Different levels of testing were performed to ensure the correctness and reliability of the system.

4.2.1 Unit Testing

This consists of testing the various components of the system.

Components to be Tested

- Login Component
- Product Management Component
- Alerts Component

Approach

- Testing individual functions.
- Inputting data and validating the output.

Tools Used

Python testing tools (manual for this project)

Advantages

- Bug detection at an early stage.
- Simplifies debugging.
- Enhances code reliability.

4.2.2 Integration Testing

Testing for integration involves checking the interaction of several modules.

Application

- Integration of front-end module with back-end module.

- Integration of back-end module with database module.

Example

- Creation of a new record through the user interface and verification if the record is created in the database.

Advantages

- Helps identify problems in the interface.
- Assures smooth data transfer between modules.

4.2.3 System Testing

System Testing performs testing on the whole system.

Scope of Testing

- User login facility
- Inventory Management System
- Stock alerting

Approach

- End to end testing
- Simulating user scenario

Advantages

- System is tested for requirement satisfaction
- Workflow validation

4.2.4 User Interface Testing

Tests have been performed to ensure that input can be provided efficiently within the system, that processes can be performed, and that results can be obtained in an effective manner. The display field that displays data encoding, decoded data, and data compression was tested for efficiency.

4.3 Test Cases

Test of user acceptance is carried out by the user to determine whether the system meets the user's requirements.

Process

- Use of the system by the Admin
- Performs real-world activities
- Provides feedback

Example

- Testing of the product addition process
- Testing of the alarm system

Benefits

- Meets the requirements of user satisfaction
- Performed usability testing

Test reports for Unit Test Cases and System Test Cases

Test Case ID	Module	Input	Expected Output	Actual Output	Status
TC01	Login	Valid credentials	Login success	Login success	Pass
TC02	Login	Invalid credentials	Error message	Error message	Pass
TC03	Add Product	Valid data	Product added	Product added	Pass
TC04	Update Product	Change quantity	Updated quantity	Updated	Pass
TC05	Delete Product	Product ID	Product removed	Removed	Pass
TC06	Alert System	Quantity < 5	Alert shown	Alert shown	Pass

Table 3

Unit Test Case Table

SYSTEM TEST CASE REPORT

System testing checks the complete flow of the website

Test Case ID	Scenario	Input	Expected Result	Status
ST01	Login System	Correct credentials	Access granted	Pass
ST02	Inventory Add	Add product	Product displayed	Pass
ST03	Inventory Update	Update quantity	Updated data shown	Pass
ST04	Low Stock Alert	Quantity < threshold	Red alert displayed	Pass
ST05	Multi-Shop	Switch shop	Correct data displayed	Pass
ST06	Export Data	Click export	CSV file downloaded	Pass

System Test Case Table

Table: System-Level Functionality Testing

Table 4

5.SYSTEM SECURITY MEASURES

Authentication and Access control are major elements of system security, ensuring that only authorized persons are able to gain access to certain system functions. In the Smart Inventory Management System, session authentication is employed in such a way that both login and logout occur based on the username and password of the individual. These details will be checked against those stored in the database, and should they match, access will be granted; otherwise, the person will get an error message. In the present version of the system, access control by administrators is used and various functions such as adding, editing products and data export can be performed.

Security of the information storage and data encryption are very important for securing stored information. First of all, there is a guarantee that passwords are stored in an encrypted way, which means that there will be no problems even in case of some kind of accessing database. Second of all, validation of input data is provided by system, which can help to prevent some kind of manipulations of data. And finally, all data is saved in MySQL database, having all required constraints.

The system provides secure communication between client and server, and uses HTTP or HTTPS protocol. Preferably, HTTPS protocol should be used, since it encrypts communication process and avoids any attempts to intercept transmitted data. There are unique session IDs, which are assigned by server to each session and expire upon logging out from the account. Moreover, every request sent to the server is always validated on server side to prevent API abuse.

To start with, application security will protect the application software from any security threat from either internal or external sources. Input validation will ensure that the invalid input is not accepted. On the other hand, parameterization will prevent any chances of SQL injection. The error handling process will present the user friendly messages while protecting the technical details. Session security will ensure that there are no cases of hijacking of sessions. These steps will increase security and reliability.

To begin with, the aim of device security is to provide security at the hardware and environment levels. This involves restricting access to only authorized persons, and thus the computer cannot be accessed by unauthorized users.

Lastly, backup and recovery security will prevent data loss through crashes and disasters. This can be done by making periodic backups of the data stored in the database to save data using MySQL. Administrators can choose to save data manually in CSV formats.

The various security features of the system have the following benefits, including data security, prevention of any access by an unknown individual, and increased system reliability. However, the present system suffers from several weaknesses like weak authentication processes, lack of two-factor authentication, poor encryption techniques, and absence of firewall technologies. All these issues can be corrected in future upgrades.

Some of the future developments in the area of security can include two-factor authentication, biometrics, enhanced encryption algorithms, cloud security services, and artificial intelligence-enabled threat detection. With these developments, the security of the system would improve considerably.

Security systems are used in several areas today such as banking systems, e-commerce platforms, healthcare systems, and inventory management systems. For instance, if an unauthorized user tries to login to the system, then the system does not accept his password, returns him an error message, and stops him from logging in.

There are many benefits of system security mechanisms like maintaining confidentiality of the information, preventing any sort of unauthorized access and improving reliability of the system itself. But there are still some drawbacks of the current system like lack of two-factor authentication, absence of advanced encryption mechanisms and firewall techniques, etc. These drawbacks can be solved in upcoming iterations.

In the upcoming iterations of system security, some of the important things that can be improved are 2FA, biometric authentication mechanisms, better encryption techniques, cloud-based security mechanisms and artificial intelligence-based security detection systems. These would help in enhancing security of the system.

These types of security measures are implemented practically for a wide range of purposes like banking applications, e-commerce applications, healthcare management applications, inventory management applications, etc. If an unauthorized person tries to enter into the system, the system simply rejects the invalid entry and shows error message to the attacker.

There are various benefits associated with the use of the security measures which include data confidentiality, prevention of any form of illegal access, and enhanced system reliability. On the other hand, some of the flaws which currently affect the security system include simple authentication techniques, no use of two-factor authentication, less sophisticated encryption methods, and lack of a more efficient firewall mechanism.

Possible improvements that can be made in future when upgrading the security system include two-factor authentication (2FA), biometric authentication, sophisticated encryption methods, cloud security solutions, and AI-based systems that can detect threats. Implementation of these measures will improve the system's security in a major way.

The security measures are widely employed in the industry in applications such as banking systems, e-commerce system, healthcare management systems, and inventory management systems among others. For instance, where a user who is not authorized tries to log in, the system denies access through displaying the appropriate message.

6.COST ESTIMATION

Cost estimation is an important part of software development that helps in predicting the total cost, effort, and time required to complete the project. In this project, both **basic cost estimation** and the **COCOMO model** are used.

Hardware Cost

Hardware cost includes the physical devices required for development and execution of the system.

Component	Description	Cost (INR)
Computer/Laptop	Development system	₹50,000
Internet Connection	Required for development & testing	₹1,000/month
Storage Devices	Backup storage	₹2,000

Table 5

Total Hardware Cost

Approx. **₹53,000**

Software Cost

Most tools used in this project are open-source, so software cost is minimal.

Software	Purpose	Cost
Python (Flask)	Backend	Free
MySQL	Database	Free
VS Code	Development IDE	Free
Browser	Testing	Free

Table 6

Total Software Cost

₹0 (Open Source Tools)

Development Cost

Development cost includes human effort and time spent on the project.

Resource	Duration	Cost
Developer (Student)	3 months	₹15,000/month

Table 7

Total Development Cost

₹45,000

COST ESTIMATION MODEL (Detailed – COCOMO MODEL)

COCOMO is a model for estimating the effort, duration, and cost of developing a software project according to its size.

10. Selecting the Type of Project

COCOMO has three types:

- Organic
- Semi-Detached
- Embedded

Selected Type of Project: Organic

Reasons:

- Small project
- Designed by a small team
- Low complexity

LOC (Lines of Code) Estimation

Estimated lines of code for the project:

Module	LOC
Frontend	1500
Backend	2000
Database	500

Calculate Effort (E)

COCOMO Formula:

$$E = a \times (KLOC)^b$$

For Organic:

- $a = 2.4$
- $b = 1.05$

Calculation

$$E = 2.4 \times (4)^{1.05}$$

$$E \approx 2.4 \times 4.28 = 10.27 \text{ person-months}$$

Calculate Development Time (D)

For Organic:

- $c = 2.5$
- $d = 0.38$

Calculation

$$D = 2.5 \times (10.27)^{0.38}$$
$$D \approx 2.5 \times 2.43 = 6.07 \text{ months}$$

Team Size

$$= \frac{10.27}{6.07} \approx 1.7$$

Cost Calculation

Assuming:

- Cost per person/month = ₹15,000

$$\begin{aligned} \text{Total Cost} &= \text{Effort} \times \text{Cost per person} \\ &= 10.27 \times 15,000 \\ &\approx ₹1,54,050 \end{aligned}$$

7.REPORTS

In any inventory management, reports act as an important component since they are used to structure the data to enable further analysis. In the Smart Inventory Management System, reports are generated automatically by obtaining data from real-time database data. The reports help the administrator to monitor inventory, manage the product, and identify potential under and over stocking situations. The function of reports in inventory management is aimed at presenting inventory data overview, decision-making, inventory availability, recording of data, and identifying issues such as low stocks at times.

Various kinds of reports are generated to cater for various methods of managing the inventory. The first one is the inventory report, which presents data on all the products present in the system. This kind of report provides product id, name, number of stocks, price, and relevant shops. It acts effectively when the manager wants to examine inventory status. The second kind is inventory below threshold report, which contains information on the products where the number of stocks have been reduced below threshold level.

Moreover, it is also possible to generate shop-wise reports that comprise all the information regarding the stock kept at the respective shop. Shop-wise reports include the name of the shop, the items that are there in the respective shop, and the number of such items. It would be easier to check up on how efficiently all the shops are performing. Besides, the report with information related to an item includes the code of the item, name of the item, number of items, and minimum stock required. First of all, all the raw data are arranged properly. Then, the report is created using tabular form.

The process of report generation consists of a number of stages for the provision of effective results. First of all, the data is obtained from the MySQL database. The next stage includes sorting and structuring data. The following step means preparing the report by arranging it in the form of a table. Finally, the generated report may be viewed on the application interface or saved as a CSV file. Reports usually have a structure with such columns as product ID, product name, quantity of units in the warehouse, cost, and store name. As for low-stock reports, they contain product names, as well as the quantity and threshold of products.

Several characteristic features can be mentioned as defining in regard to the report system offered here. Another advantage of the system is its possibility to export data as a table.

The following is an overview of some of the advantages associated with the use of reports in the system. They enhance the decision-making process as they offer timely and accurate information that is needed in this case. They enable users to manage their inventory effectively as they are capable of tracking their inventory easily. They save users' time since the data analysis is done automatically. Moreover, the reports help users get reliable information that they can use when auditing their inventory. Unfortunately, the present reporting system comes with a number of disadvantages.

There are certain developments that can be introduced to make the reports more useful to users. Some of the features include a provision for creating graphical reports using different chart types. There is a need for having advanced analytics dashboard that can help in offering deeper data analysis. Other important features include automatic generation of the reports and e-mail notification as well. Users should also have an option of exporting the reports in PDF format.

The future improvements of the reporting system can comprise more advanced capabilities, such as graphical reports, which will be generated using charts and graphs. Another potential improvement can relate to the creation of a powerful analysis tool that is capable of providing a greater level of understanding of the dynamics of the inventory. Capabilities such as automated report generation and notifications by email can be developed to keep users up-to-date on everything. It is also possible to implement a report export feature that uses PDF format. The system can also provide the capability to generate reports based on filters.

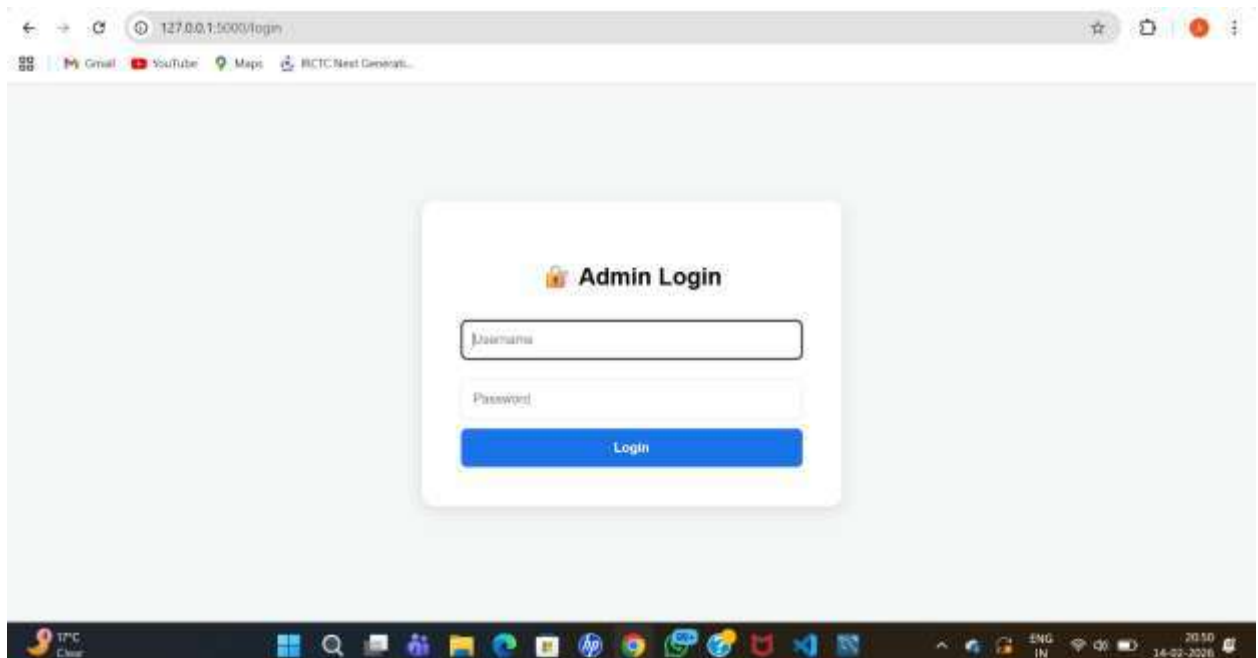


Fig 9 Admin page

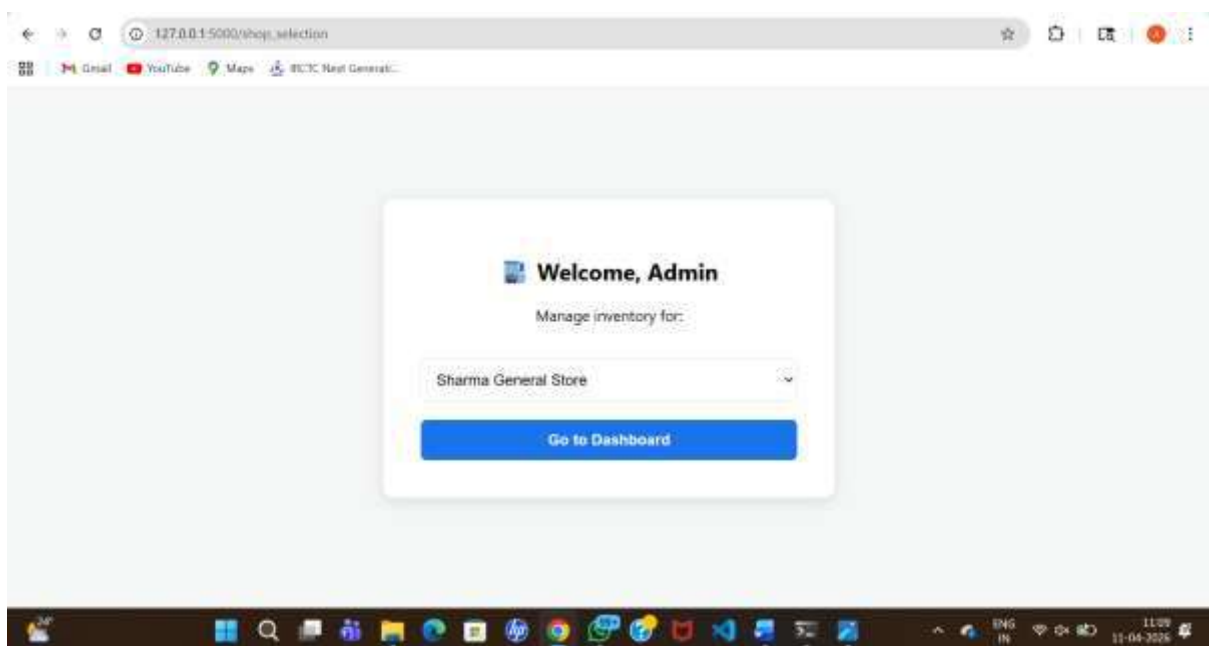


Fig.10 Selection of shops

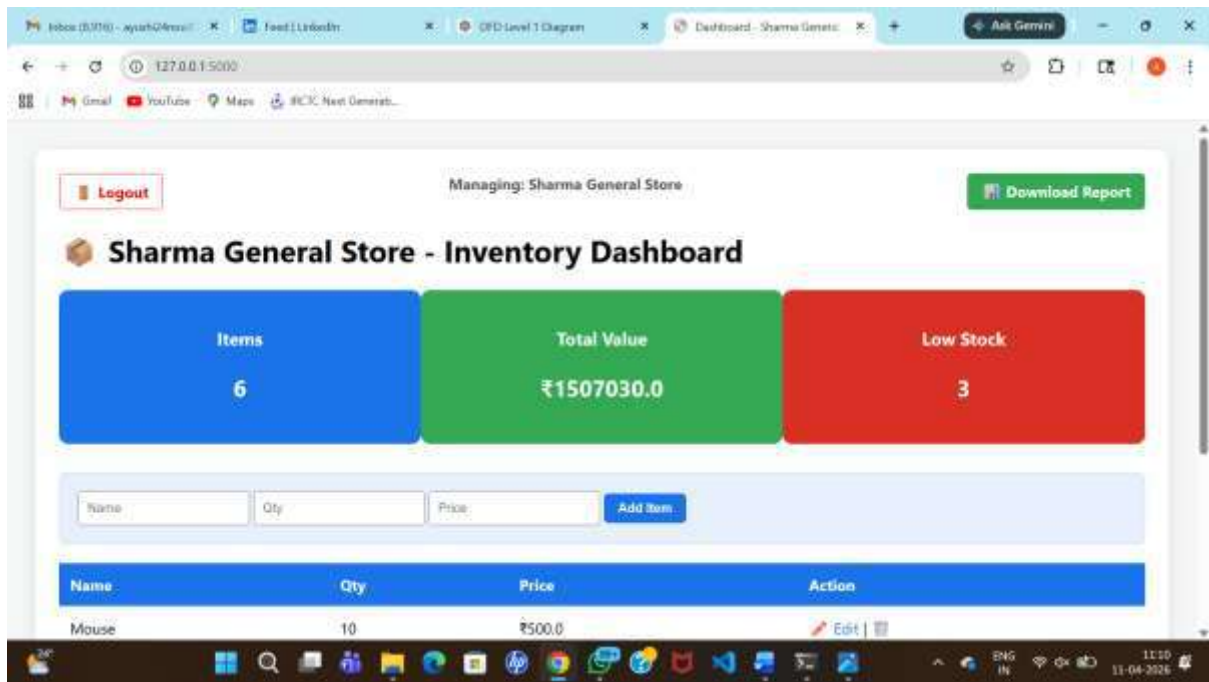


Fig.11 Dashboard

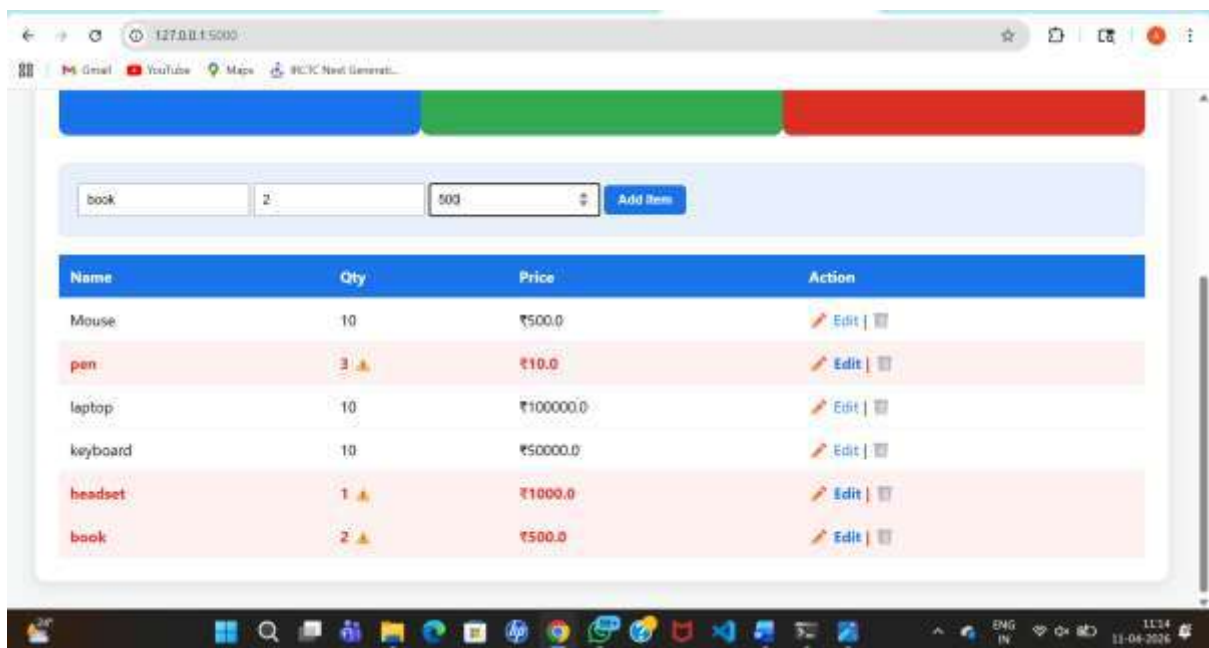


Fig.12 List of items

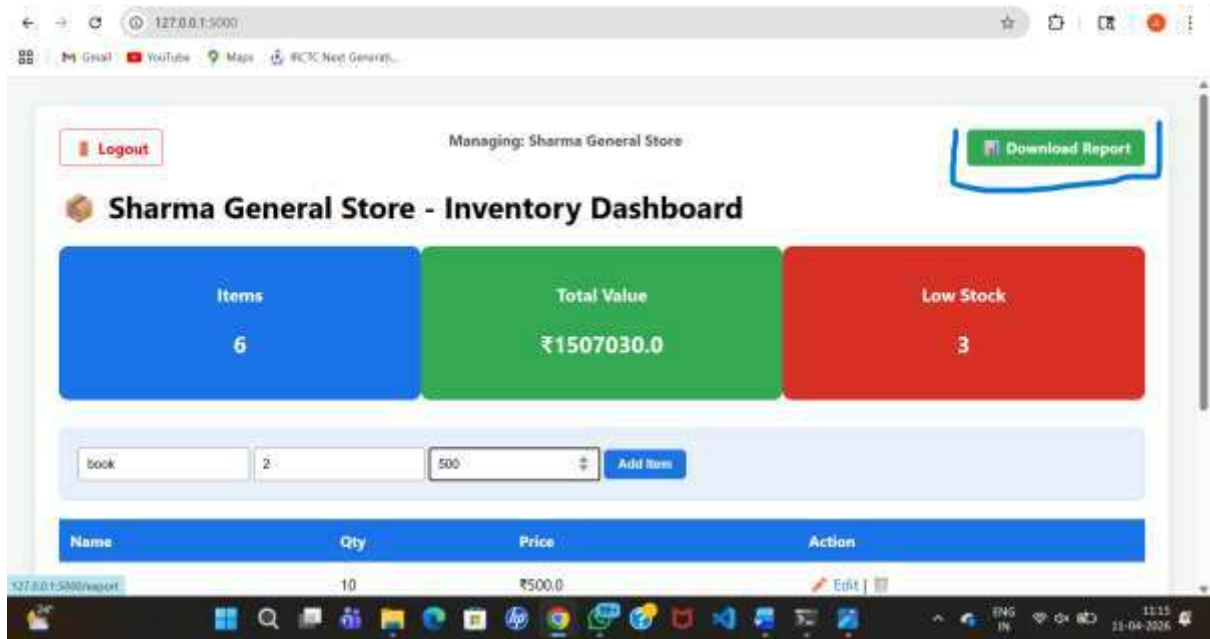


Fig.13 Download Report Icon

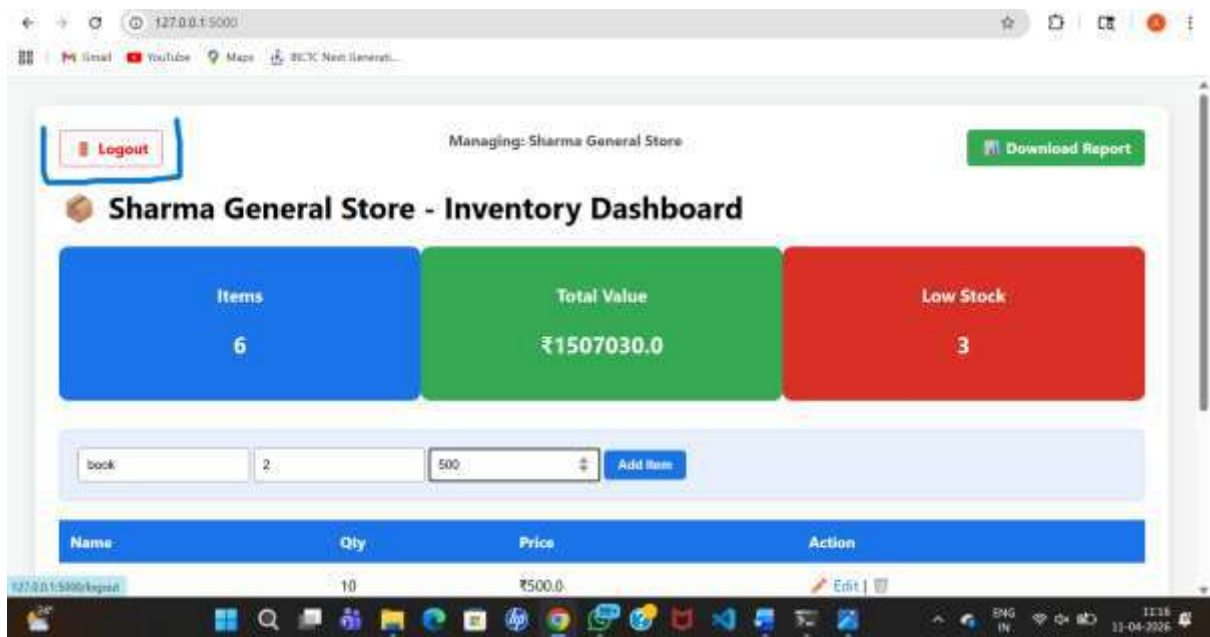


Fig.14 Logout icon

8.FUTURE SCOPE AND ENHANCEMENT

The Inventory Management System that has been designed through this project has proven its capabilities regarding efficient inventory management. Nevertheless, the advancements in technology and increased demands of business entities allow room for enhancement in the future. The future scope of the Inventory Management System will include improvements in terms of system efficiency, scalability, usability, and application of innovative technologies in order to enhance efficiency. The present system has met all necessary requirements for efficient functioning; however, it still has some drawbacks such as the limitation in the number of available user roles, simple report generation capabilities, and lack of automation. Thus, future enhancements are required to enhance the system in terms of overcoming the mentioned drawbacks.

2. Improvements Needed in the Future: Though the system works according to requirements, there are some shortcomings like limitations in user roles, easy reporting, and no automation. Hence, modifications are required to improve the efficiency of the system, improve the user interface, make the system work for big organizations, apply new technologies, and generate sophisticated reports.

3. Role-Based Authentication: The current system only provides the facility to log in as an administrator. In the future, role-based authentication could be applied to create various user roles like that of administrator, manager, and staff.

4. Mobile Application Development: This can be achieved through developing an application on either Android or iOS platform. This will enhance the accessibility of the inventory from any place at anytime and allow for updates on the inventory.

5. Implementation of Barcode/QR Code Technology: Integration of this technology can be done into the system to facilitate efficient entry of products in addition to eliminating manual mistakes.

6. Implementation of Cloud Computing: It entails allowing remote access to data, automatic back up and increased scalability of the system. With this feature, large data volumes and users will be handled effectively.

7. Graphical Reports Sales Trend Analysis and Demand Forecasting: Implementation of future improvements that include these features will increase the effectiveness of this system in decision making.

8. Integration of Artificial Intelligence (AI): This is aimed at facilitating prediction and automation of inventory needs and smart alerts for the user.

9. Notification System Enhancement: Enhancing the notification process can be achieved through incorporation of emails alerts, SMS and Push notifications.

10. Integration with Billing Software: Integration with the billing and sales software can be done to facilitate the updating of inventory automatically post sales transactions.

11. Improved Security Measures: Improved security measures for the software can be incorporated by means of providing two-factor authentication, encryption of data, and logging of activities to secure the software from being hacked.

12. Multi-language Interface: A multi-language interface for the software can be included based on the geographical locations of the users.

13. Working of the Software without Internet Connection: The designing of the software can incorporate an offline mode of working.

14. Future Dashboards: Future dashboards will include graphical representation and real-time analysis.

In terms of security, there is a possibility to incorporate such elements as 2FA, data encryption, and activity log. These improvements will ensure the safety of the system against any illegal intrusion and guarantee the security of confidential data. Besides, the ability to operate the system using multiple languages may make the product more accessible for people of different parts of the globe. Additionally, the introduction of offline operation is possible; all the data is synchronized automatically once the internet connection is established.

In terms of the improvements aimed at providing a rich user experience, there are several possibilities. First, a real-time analysis feature could serve useful when analyzing relevant data. Second, a graphical representation of data will contribute to a better understanding of the

information being presented. Third, it is crucial to implement the functionality which will allow personalizing the appearance of the interface.

One of the major concerns regarding the creation of the system is its capacity to operate with a large number of users, branches/warehouses or transactions. In other words, scalability is one of the key characteristics of the future system.

15. Scalability of the System: The system can be designed in a manner that enables it to handle many users, branches, and transactions. With the scalability of the system, it would ensure its efficiency despite the growth of the organization.

16. Benefits of the Future Enhancements: Through these future enhancements, the system would become more efficient, user-friendly, secure, decision-making friendly, and with modern technology incorporated into the system.

9. Appendices

9.1 coding

```
<!DOCTYPE html>

<html>

<head>

  <title>Dashboard - {{ session['current_shop'] }}</title>

  <style>

    body { font-family: 'Segoe UI', sans-serif; background: #f4f7f6; margin: 30px; }

    .container { background: white; padding: 25px; border-radius: 12px; box-shadow: 0 4px 15px. rgba(0,0,0,0.1); }

    .top-bar { display: flex; justify-content: space-between; margin-bottom: 20px; }

    .card-container { display: flex; justify-content: space-between; margin-bottom: 30px; }

    .card { padding: 20px; border-radius: 10px; width: 30%; text-align: center; color: white; font-weight: bold; }

    .form-box { background: #e8f0fe; padding: 20px; border-radius: 8px; margin-bottom: 20px; border: 1px solid #c2d7ff; }

    table { width: 100%; border-collapse: collapse; }

    th, td { padding: 12px; border-bottom: 1px solid #eee; text-align: left; }

    th { background: #1a73e8; color: white; }

    .btn { padding: 8px 15px; border-radius: 5px; text-decoration: none; font-weight: bold; cursor: pointer; border: none; }

    .low-stock-row { background-color: #fff1f0; color: #d93025; font-weight: bold; }

  </style>

</head>

<body>

  <div class="container">

    <div class="top-bar">

      <a href="/logout" class="btn" style="border: 1px solid red; color: red;">  Logout</a>

      <span style="font-weight: bold; color: #555;">Managing: {{ session['current_shop'] }}</span>

      <a href="/export" class="btn" style="background: #34a853; color: white;">  Download Report</a>

    </div>

  </div>

</body>

</html>
```

</div>

<h1> 📦 {{ session['current_shop'] }} - Inventory Dashboard</h1>

<div class="card-container">

<div class="card" style="background: #1a73e8;"><h3>Items</h3><p style="font-size: 24px;">{{ total_items }}</p></div>

<div class="card" style="background: #34a853;"><h3>Total Value</h3><p style="font-size: 24px;">₹{{ total_value }}</p></div>

<div class="card" style="background: #d93025;"><h3>Low Stock</h3><p style="font-size: 24px;">{{ low_stock_count }}</p></div>

</div>

<div class="form-box">

<form action="/add" method="POST">

<input type="text" name="name" placeholder="Name" required style="padding:8px;">

<input type="number" name="qty" placeholder="Qty" required style="padding:8px;">

<input type="number" name="price" placeholder="Price" required style="padding:8px;">

<button type="submit" class="btn" style="background: #1a73e8; color: white;">Add Item</button>

</form>

</div>

<table>

<thead><tr><th>Name</th><th>Qty</th><th>Price</th><th>Action</th></tr></thead>

<tbody>

{% for item in items %}

<tr class="{{ 'low-stock-row' if item.quantity < 5 else ' ' }}">

```

<td>{{ item.name }}</td>

<td>{{ item.quantity }} {% if item.quantity < 5 %} ⚠️ {% endif %}</td>

<td>₹{{ item.price }}</td>

<td>

    <a href="/edit/{{ item.id }}" style="color: #1a73e8; text-decoration: none;">✎
Edit</a> |

    <a href="/delete/{{ item.id }}" style="color: red; text-decoration: none;"
onclick="return confirm('Delete?')">🗑️ </a>

</td>

</tr>

{% endfor %}

</tbody>

</table>

</div>

</body>

</html>

```

```

import tkinter as tk

from tkinter import messagebox

import mysql.connector

# Database connection function
def get_db():

```

```

return mysql.connector.connect(
    host="localhost",
    user="root",
    password="NayaPassword123",
    database="inventory_db"
)

def add_item_gui():
    name = entry_name.get()
    qty = entry_qty.get()
    price = entry_price.get()

    if name and qty and price:
        conn = get_db()
        cursor = conn.cursor()
        cursor.execute("INSERT INTO products (name, quantity, price) VALUES (%s, %s, %s)",
            (name, qty, price))
        conn.commit()
        conn.close()
        messagebox.showinfo("Success", f"{name} added to inventory!")
    else:
        messagebox.showwarning("Input Error", "Please fill all fields")

# Main Window Setup
root = tk.Tk()
root.title("Smart Inventory System")
root.geometry("300x400")

tk.Label(root, text="Product Name").pack()

```

```

entry_name = tk.Entry(root)
entry_name.pack()

tk.Label(root, text="Quantity").pack()
entry_qty = tk.Entry(root)
entry_qty.pack()

tk.Label(root, text="Price").pack()
entry_price = tk.Entry(root)
entry_price.pack()

tk.Button(root, text="Add to Database", command=add_item_gui).pack(pady=20)

root.mainloop()

import mysql.connector

def get_db_connection():
    return mysql.connector.connect(
        host="localhost",
        user="root",
        password="NayaPassword123", # Your confirmed password
        database="inventory_db"    # Your database name
    )

def add_product(name, qty, price):
    try:

```

```

    conn = get_db_connection()
    cursor = conn.cursor()
    sql = "INSERT INTO products (name, quantity, price) VALUES (%s, %s, %s)"
    cursor.execute(sql, (name, qty, price))
    conn.commit()
    print(f"--- Added {name} to inventory! ---")
finally:
    cursor.close()
    conn.close()

def view_inventory():
    try:
        conn = get_db_connection()
        cursor = conn.cursor()
        cursor.execute("SELECT * FROM products")
        rows = cursor.fetchall()
        print("\n--- Current Inventory ---")
        print("ID | Name | Quantity | Price")
        for row in rows:
            print(f"{row[0]} | {row[1]} | {row[2]} | ${row[3]}")
    finally:
        cursor.close()
        conn.close()

# TEST IT OUT
add_product("Keyboard", 15, 45.00)
view_inventory()

from flask import Flask, render_template, request, redirect, url_for, session, Response

```

```

import mysql.connector
import csv
import io

app = Flask(__name__)
app.secret_key = "multi_shop_secret_key"

def get_db():
    return mysql.connector.connect(
        host="localhost",
        user="root",
        password="NayaPassword123",          # Aapka password
        database="inventory_db"
    )

@app.route('/login', methods=['GET', 'POST'])
def login():
    error = None

    if request.method == 'POST':
        username, password = request.form['username'], request.form['password']

        conn = get_db()
        cursor = conn.cursor(dictionary=True)
        cursor.execute("SELECT * FROM users WHERE username = %s AND password = %s",
            (username, password))
        user = cursor.fetchone()
        conn.close()

        if user:
            session['logged_in'] = True
            return redirect(url_for('shop_page'))          # after Login Shop Selection

```

```

        error = "Invalid Credentials!"

    return render_template('login.html', error=error)

@app.route('/shop_selection')
def shop_page():
    if not session.get('logged_in'): return redirect(url_for('login'))

    conn = get_db()

    cursor = conn.cursor(dictionary=True)

    cursor.execute("SELECT * FROM shops") ;      # MySQL shops

    all_shops = cursor.fetchall()

    conn.close()

    return render_template('shop_selection.html', shops=all_shops)

@app.route('/select_shop', methods=['POST'])
def select_shop():
    session['current_shop'] = request.form['shop_name']      # to save a Shop name

    return redirect(url_for('index'))

@app.route('/')
def index():
    if not session.get('logged_in') or 'current_shop' not in session:
        return redirect(url_for('login'))

    conn = get_db()

    cursor = conn.cursor(dictionary=True)

    cursor.execute("SELECT * FROM products")

    inventory = cursor.fetchall()

    total_items = len(inventory)

```

```

total_value = sum(item['quantity'] * item['price'] for item in inventory)
low_stock_count = sum(1 for item in inventory if item['quantity'] < 5)
conn.close()

return render_template('index.html', items=inventory, total_items=total_items,
total_value=total_value, low_stock_count=low_stock_count)

```

```

@app.route('/logout')
def logout():
    session.clear()
    return redirect(url_for('login'))

```

CRUD aur Export Routes (Same as before)

```

@app.route('/add', methods=['POST'])
def add_product():
    if not session.get('logged_in'): return redirect(url_for('login'))
    name, qty, price = request.form['name'], request.form['qty'], request.form['price']
    conn = get_db()
    cursor = conn.cursor()
    cursor.execute("INSERT INTO products (name, quantity, price) VALUES (%s, %s, %s)",
(name, qty, price))
    conn.commit()
    conn.close()
    return redirect(url_for('index'))

```

```

@app.route('/delete/<int:id>')
def delete_product(id):
    if not session.get('logged_in'): return redirect(url_for('login'))
    conn = get_db()
    cursor = conn.cursor()
    cursor.execute("DELETE FROM products WHERE id = %s", (id,))

```

```
conn.commit()

conn.close()

return redirect(url_for('index'))
```

```
@app.route('/edit/<int:id>')
```

```
def edit_product(id):
```

```
    if not session.get('logged_in'): return redirect(url_for('login'))
```

```
    conn = get_db()
```

```
    cursor = conn.cursor(dictionary=True)
```

```
    cursor.execute("SELECT * FROM products WHERE id = %s", (id,))
```

```
    product = cursor.fetchone()
```

```
    conn.close()
```

```
    return render_template('edit.html', product=product)
```

```
@app.route('/update', methods=['POST'])
```

```
def update_product():
```

```
    if not session.get('logged_in'): return redirect(url_for('login'))
```

```
    id, name, qty, price = request.form['id'], request.form['name'], request.form['qty'],
request.form['price']
```

```
    conn = get_db()
```

```
    cursor = conn.cursor()
```

```
    cursor.execute("UPDATE products SET name=%s, quantity=%s, price=%s WHERE
id=%s", (name, qty, price, id))
```

```
    conn.commit()
```

```
    conn.close()
```

```
    return redirect(url_for('index'))
```

```
@app.route('/export')
```

```
def export_csv():
```

```
    if not session.get('logged_in'): return redirect(url_for('login'))
```

```

conn = get_db()
cursor = conn.cursor(dictionary=True)
cursor.execute("SELECT name, quantity, price FROM products")
items = cursor.fetchall()
conn.close()
output = io.StringIO()
writer = csv.writer(output)
writer.writerow(['Product Name', 'Quantity', 'Price'])
for item in items:
    writer.writerow([item['name'], item['quantity'], item['price']])
output.seek(0)
return Response(output.getvalue(), mimetype="text/csv", headers={"Content-disposition":
"attachment; filename=report.csv"})

if __name__ == '__main__':
    app.run(debug=True)

```

Shop Selection code :

```

<!DOCTYPE html>
<html>
<head>
    <title>Select Shop</title>
    <style>
        body { font-family: 'Segoe UI', sans-serif; background: #f4f7f6; display: flex; justify-
content: center; align-items: center; height: 100vh; margin: 0; }
        .selection-card { background: white; padding: 40px; border-radius: 12px; box-shadow: 0
4px 15px rgba(0,0,0,0.1); width: 400px; text-align: center; }
        select { width: 100%; padding: 12px; margin: 20px 0; border: 1px solid #ddd; border-
radius: 6px; font-size: 16px; }

```

```
button { width: 100%; padding: 12px; background: #1a73e8; color: white; border: none;
border-radius: 6px; cursor: pointer; font-weight: bold; font-size: 16px; }
```

```
button:hover { background: #1557b0; }
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="selection-card">
```

```
<h2>🏪 Welcome, Admin</h2>
```

```
<p>Manage inventory for:</p>
```

```
<form action="/select_shop" method="POST">
```

```
<select name="shop_name">
```

```
{% for shop in shops %}
```

```
<option value="{{ shop.shop_name }}">{{ shop.shop_name }}</option>
```

```
{% endfor %}
```

```
</select>
```

```
<button type="submit">Go to Dashboard</button>
```

```
</form>
```

```
</div>
```

```
</body>
```

```
</html>
```

LOGIN TABLE CODE:

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
<title>Login - Inventory System</title>
```

```
<style>
```

```
body { font-family: sans-serif; background: #f4f7f6; display: flex; justify-content: center;
align-items: center; height: 100vh; margin: 0; }
```

```
.login-card { background: white; padding: 40px; border-radius: 12px; box-shadow: 0 4px 15px rgba(0,0,0,0.1); width: 350px; text-align: center; }
```

```
input { width: 100%; padding: 12px; margin: 10px 0; border: 1px solid #ddd; border-radius: 6px; box-sizing: border-box; }
```

```
button { width: 100%; padding: 12px; background: #1a73e8; color: white; border: none; border-radius: 6px; cursor: pointer; font-weight: bold; }
```

```
.error { color: red; margin-bottom: 10px; font-size: 0.9em; }
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<div class="login-card">
```

```
<h2>🔒 Admin Login</h2>
```

```
{% if error %} <div class="error">{{ error }}</div> {% endif %}
```

```
<form action="/login" method="POST">
```

```
<input type="text" name="username" placeholder="Username" required>
```

```
<input type="password" name="password" placeholder="Password" required>
```

```
<button type="submit">Login</button>
```

```
</form>
```

```
</div>
```

```
</body>
```

```
</html>
```

DATABASE CODE:

```
import mysql.connector
```

```
def get_db_connection():
```

```
    return mysql.connector.connect(
```

```
        host="localhost",
```

```
        user="root",
```

```
        password="NayaPassword123", # Aapka password
```

```

        database="inventory_db"
    )

def add_product():
    name = input("Product Name: ")
    qty = int(input("Quantity: "))
    price = float(input("Price: "))

    conn = get_db_connection()
    cursor = conn.cursor()

    cursor.execute("INSERT INTO products (name, quantity, price) VALUES (%s, %s, %s)",
    (name, qty, price))

    conn.commit()

    print("☑ Item added successfully!")

    conn.close()

def smart_view():
    conn = get_db_connection()
    cursor = conn.cursor()

    # Sabhi items dikhayega
    cursor.execute("SELECT * FROM products")

    items = cursor.fetchall()

    print("\n--- Current Inventory ---")

    for i in items:
        status = "⚠ LOW STOCK" if i[2] < 5 else "☑ OK"

        print(f"ID: {i[0]} | Name: {i[1]} | Qty: {i[2]} | Price: {i[3]} | Status: {status}")

    conn.close()

```

MAIN MENU LOOP

while True:

print("\n1. Add Product 2. View Inventory & Alerts 3. Exit")

choice = input("Select Option: ")

if choice == '1':

add_product()

elif choice == '2':

smart_view()

elif choice == '3':

break

<!DOCTYPE html>

<html>

<head><title>Edit Product</title></head>

<body>

<h2>✎ Edit Product</h2>

<form action="/update" method="POST">

<input type="hidden" name="id" value="{{ product.id }}">

Name: <input type="text" name="name" value="{{ product.name }}" required>

Qty: <input type="number" name="qty" value="{{ product.quantity }}" required>

Price: <input type="number" name="price" value="{{ product.price }}" required>

<button type="submit">Update</button>

</form>

</body>

</html>

from flask import Flask, render_template, request, redirect, url_for, session, Response

```

import mysql.connector
import csv
import io

app = Flask(__name__)
app.secret_key = "multi_shop_secret_key"

def get_db():
    return mysql.connector.connect(
        host="localhost",
        user="root",
        password="NayaPassword123", # Aapka password
        database="inventory_db"
    )

@app.route('/login', methods=['GET', 'POST'])
def login():
    error = None

    if request.method == 'POST':
        username, password = request.form['username'], request.form['password']

        conn = get_db()
        cursor = conn.cursor(dictionary=True)
        cursor.execute("SELECT * FROM users WHERE username = %s AND password = %s",
            (username, password))
        user = cursor.fetchone()
        conn.close()

        if user:
            session['logged_in'] = True
            return redirect(url_for('shop_page')) # Login ke baad Shop Selection

```

```

        error = "Invalid Credentials!"

    return render_template('login.html', error=error)

@app.route('/shop_selection')
def shop_page():
    if not session.get('logged_in'): return redirect(url_for('login'))

    conn = get_db()

    cursor = conn.cursor(dictionary=True)

    cursor.execute("SELECT * FROM shops") # MySQL se shops lana
    all_shops = cursor.fetchall()

    conn.close()

    return render_template('shop_selection.html', shops=all_shops)

@app.route('/select_shop', methods=['POST'])
def select_shop():
    session['current_shop'] = request.form['shop_name'] # Shop name save karna
    return redirect(url_for('index'))

@app.route('/')
def index():
    if not session.get('logged_in') or 'current_shop' not in session:
        return redirect(url_for('login'))

    conn = get_db()

    cursor = conn.cursor(dictionary=True)

    cursor.execute("SELECT * FROM products")

    inventory = cursor.fetchall()

    total_items = len(inventory)

```

```

total_value = sum(item['quantity'] * item['price'] for item in inventory)
low_stock_count = sum(1 for item in inventory if item['quantity'] < 5)
conn.close()

return render_template('index.html', items=inventory, total_items=total_items,
total_value=total_value, low_stock_count=low_stock_count)

```

```

@app.route('/logout')
def logout():
    session.clear()
    return redirect(url_for('login'))

```

CRUD aur Export Routes (Same as before)

```

@app.route('/add', methods=['POST'])
def add_product():
    if not session.get('logged_in'): return redirect(url_for('login'))
    name, qty, price = request.form['name'], request.form['qty'], request.form['price']
    conn = get_db()
    cursor = conn.cursor()
    cursor.execute("INSERT INTO products (name, quantity, price) VALUES (%s, %s, %s)",
(name, qty, price))
    conn.commit()
    conn.close()
    return redirect(url_for('index'))

```

```

@app.route('/delete/<int:id>')
def delete_product(id):
    if not session.get('logged_in'): return redirect(url_for('login'))
    conn = get_db()
    cursor = conn.cursor()
    cursor.execute("DELETE FROM products WHERE id = %s", (id,))

```

```
conn.commit()

conn.close()

return redirect(url_for('index'))
```

```
@app.route('/edit/<int:id>')
```

```
def edit_product(id):
```

```
    if not session.get('logged_in'): return redirect(url_for('login'))
```

```
    conn = get_db()
```

```
    cursor = conn.cursor(dictionary=True)
```

```
    cursor.execute("SELECT * FROM products WHERE id = %s", (id,))
```

```
    product = cursor.fetchone()
```

```
    conn.close()
```

```
    return render_template('edit.html', product=product)
```

```
@app.route('/update', methods=['POST'])
```

```
def update_product():
```

```
    if not session.get('logged_in'): return redirect(url_for('login'))
```

```
    id, name, qty, price = request.form['id'], request.form['name'], request.form['qty'],
request.form['price']
```

```
    conn = get_db()
```

```
    cursor = conn.cursor()
```

```
    cursor.execute("UPDATE products SET name=%s, quantity=%s, price=%s WHERE
id=%s", (name, qty, price, id))
```

```
    conn.commit()
```

```
    conn.close()
```

```
    return redirect(url_for('index'))
```

```
@app.route('/export')
```

```
def export_csv():
```

```
    if not session.get('logged_in'): return redirect(url_for('login'))
```

```

conn = get_db()
cursor = conn.cursor(dictionary=True)
cursor.execute("SELECT name, quantity, price FROM products")
items = cursor.fetchall()
conn.close()
output = io.StringIO()
writer = csv.writer(output)
writer.writerow(['Product Name', 'Quantity', 'Price'])
for item in items:
    writer.writerow([item['name'], item['quantity'], item['price']])
output.seek(0)
return Response(output.getvalue(), mimetype="text/csv", headers={"Content-disposition":
"attachment; filename=report.csv"})

if __name__ == '__main__':
    app.run(debug=True)

from flask import Flask, render_template, request, redirect, url_for, session, Response, flash
import mysql.connector
from werkzeug.security import generate_password_hash, check_password_hash
import csv
import io

app = Flask(__name__)
app.secret_key = "multi_shop_secret_key"

```

9.2 Bibliography

Books

1. Pressman, Roger S. *Software Engineering: A Practitioner's Approach*, 7th Edition, McGraw-Hill Education.
2. Sommerville, Ian. *Software Engineering*, 10th Edition, Pearson Education.
3. Elmasri, Ramez & Navathe, Shamkant B. *Fundamentals of Database Systems*, Pearson.
4. Silberschatz, Abraham, Korth, Henry F., & Sudarshan, S. *Database System Concepts*, McGraw-Hill.

Web References

1. Official Python Documentation –
<https://docs.python.org/3/>
2. Flask Official Documentation –
<https://flask.palletsprojects.com/>
3. MySQL Official Documentation –
<https://dev.mysql.com/doc/>
4. W3Schools –
<https://www.w3schools.com/>
5. GeeksforGeeks –
<https://www.geeksforgeeks.org/>
6. Stack Overflow –
<https://stackoverflow.com/>